EMPLOYEE MANAGEMENT ANALYSIS - SQL

Database
use employee_db;

Table Schema

Master Tables - Department, Location, Job_roles

Departments
- dept_id int primary key,
- dept_name varchar(50) not null

Locations
- location_id int primary key,
- location_name varchar(50) not null

Job_roles
- role_id int primary key,
- role_name varchar(50) not null,
- min_salary decimal(10,2),
- max_salary decimal(10,2)

Other Important Tables and Schemas

Employees
- emp_id int primary key,
- emp_name varchar(100) not null,
- gender varchar(10),
- dept_id int,
- role_id int,
- location_id int,
- join_date date,
- status varchar(20) default 'Active',
- foreign key (dept_id) references Departments(dept_id),
- foreign key (role_id) references Job_roles(role_id),
- foreign key (location_id) references Locations(location_id)

Salaries
- salary_id int primary key,
- emp_id int,
- salary decimal(10,2),
- effective_date date,

- foreign key (emp_id) references Employees(emp_id)

Performance_Reviews

- review_id int primary key,
- emp_id int,
- review_year date,
- rating decimal(2,1) check (rating between 1 and 5),
- foreign key (emp_id) references Employees(emp_id)

PERFORMANCE ANALYSIS

1. **Top 10 high performers**

```
select top 10 * from Performance_review
order by rating desc ;
```

Output:

100 %

Results | Messages

	review_id	emp_id	review_year	rating
1	12	105	2023-08-08	4.7
2	6	102	2023-12-01	4.6
3	4	102	2021-02-10	4.5
4	5	102	2022-07-18	4.3
5	3	101	2023-11-05	4.2
6	17	107	2023-07-21	4.2
7	16	107	2022-03-27	4
8	2	101	2022-06-20	3.9
9	15	106	2023-12-15	3.9
10	8	103	2023-09-14	3.8

2. **Average Rating Per department**

```
select e.dept_id, round(avg(p.rating),2) as avg_per_department
from Performance_review p
left join Employees e
on e.emp_id = p.emp_id
group by e.dept_id;
```

Output:

Results | Messages

	dept_id	avg_per_department
1	1	4.17
2	2	3.43
3	3	3.5
4	4	3
5	5	4.7
6	6	4.1

3. **Employees Eligible for promotion**

```
select p.emp_id,round(p.rating,2)as rating from Performance_review p
left join Employees e
on e.emp_id = p.emp_id
where round(p.rating,2) > 4.0 and DATEDIFF(YEAR, e.join_date, GETDATE()) > 3;
```

Output:

Results | Messages

	emp_id	rating
1	101	4.2
2	102	4.5
3	102	4.3
4	102	4.6
5	105	4.7
6	107	4.2

4. **Performance Trend over years**

```
select emp_id, review_year, round(rating,2)as rating
from Performance_review
order by emp_id, review_year;
```

Output:

Results | Messages

	emp_id	review_year	rating
1	101	2021-03-15	3.5
2	101	2022-06-20	3.9
3	101	2023-11-05	4.2
4	102	2021-02-10	4.5
5	102	2022-07-18	4.3
6	102	2023-12-01	4.6

SALARY ANALYSIS

5. **Salary Distribution per department**

```
select  d.dept_name,
avg(s.salary) as avg_salary
from Salaries s
left join Employees e
on e.emp_id = s.emp_id
left join Departments d
on e.dept_id = d.dept_id
group by d.dept_name
order by avg_salary desc;
```

Output:

Results | Messages

	dept_name	avg_salary
1	Legal	1192500
2	Research and Development	956250
3	Business Analysis	907500
4	Marketing	900000
5	Logistics	833750
6	Finance	803750

6. **Highest and lowest salary per role**

```
select d.dept_name, min(s.salary) as Lowest_salary, max(s.salary) as Highest_salary
from Salaries s
left join Employees e
on e.emp_id = s.emp_id
left join Departments d
on e.dept_id = d.dept_id
group by d.dept_name;
```

Output:

Results | Messages

	dept_name	Lowest_salary	Highest_salary
1	Administration	540000	840000
2	Business Analysis	900000	915000
3	Customer Support	700000	740000
4	Finance	620000	875000
5	HR	420000	645000
6	IT	500000	750000

7. **Salary vs Performance correlation**

```
select
   round(p.rating,2) as rating,
   count(*) as total_employees,
   avg(s.salary) as avg_salary
from Salaries s
join Performance_Review p on p.emp_id = s.emp_id
group by p.rating
order by p.rating;
```

Output:

Results | Messages

	rating	total_employees	avg_salary
1	2.5	1	720000
2	2.8	1	720000
3	2.9	1	810000
4	3	2	615000
5	3.1	1	810000
6	3.2	1	620000

8. **Employees below department average salary**

```
select
    s.emp_id,
    s.salary,
    d.dept_id
from Salaries s
join Employees e on e.emp_id = s.emp_id
join Departments d on d.dept_id = e.dept_id
where s.salary < (
    select avg(s2.salary)
    from Salaries s2
    join Employees e2 on e2.emp_id = s2.emp_id
    where e2.dept_id = e.dept_id
);
```

Output:

Results | Messages

	emp_id	salary	dept_id
1	101	500000	1
2	106	420000	2
3	103	620000	3
4	104	810000	4
5	121	560000	5
6	136	520000	5

DEPARTMENT GROWTH

9. Hiring Trend Per year

```
select year(join_date)as years, count(*) as Hire_trend
from Employees
group by year(join_date);
```

Output:

Results | Messages

	years	Hire_trend
1	2016	6
2	2017	10
3	2018	12
4	2019	13
5	2020	11
6	2021	11

10. Headcount growth by department

```
select  dept_id,count(*) as Headcount, year(join_date) as years
from Employees
group by  dept_id, year(join_date)
order by dept_id;
```

Output:

Results | Messages

	dept_id	Headcount	years
1	1	1	2017
2	1	1	2018
3	1	2	2019
4	1	1	2021
5	1	1	2023
6	2	1	2018

11. Attrition rate by department

```
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS total_employees,
    SUM(CASE WHEN e.status = 'Inactive' THEN 1 ELSE 0 END) AS employees_left,
    ROUND(
        100.0 * SUM(CASE WHEN e.status = 'Inactive' THEN 1 ELSE 0 END)
        / COUNT(e.emp_id),
        2
    ) AS attrition_rate_percent
FROM Employees e
JOIN Departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY attrition_rate_percent DESC;
```

Output:

Results | Messages

	dept_name	total_employees	employees_left	attrition_rate_percent
1	Public Relations	4	1	25.000000000000
2	Quality Assurance	5	1	20.000000000000
3	Research and Development	5	1	20.000000000000
4	Sales	5	1	20.000000000000
5	Administration	5	1	20.000000000000
6	Business Analysis	5	1	20.000000000000

STRATEGIC INSIGHTS

12. Employees over due for promotion

```
select e.emp_id,e.emp_name,datediff(year,e.join_date,getdate())as experience,
round(p.rating,2) as rating, s.salary
from Employees e
join Salaries s
on s.emp_id = e.emp_id
join Performance_review p
on p.emp_id = e.emp_id;
```

Output:

Results | Messages

	emp_id	emp_name	experience	rating	salary
1	101	Amit Sharma	5	3.5	500000
2	101	Amit Sharma	5	3.9	500000
3	101	Amit Sharma	5	4.2	500000
4	102	Priya Nair	7	4.5	750000
5	102	Priya Nair	7	4.3	750000
6	102	Priya Nair	7	4.6	750000

13. High salary but low performance cases

```
select p.emp_id, round(p.rating,2) as rating, s.salary
from Performance_review p
left join Salaries s
on p.emp_id = s.emp_id
where p.rating < 3.5 and s.salary > 500000;
```

Output:

Results | Messages

	emp_id	rating	salary
1	103	3.2	620000
2	104	2.9	810000
3	104	3.1	810000
4	104	3	810000
5	108	2.5	720000
6	108	2.8	720000

14. Location wise salary comparison

```
select l.location_id,l.location_name, avg(s.salary) as avg_Salary
from Locations l
join Employees e on e.location_id = l.location_id
join Salaries s on e.emp_id = s.emp_id
group by l.location_id,l.location_name
order by avg_Salary desc;
```

Output:

Results | Messages

	location_id	location_name	avg_Salary
1	13	Chandigarh	1075000
2	11	Kochi	948750
3	10	Coimbatore	891250
4	6	Delhi	867500
5	1	Bangalore	770000
6	14	Lucknow	768750

15. Gender diversity ration

```
select round(sum(case when e.gender = 'Male' then 1 else 0 end )*100.0 / count(*),2) as Male,
round(sum(case when e.gender = 'Female' then 1 else 0 end )*100.0 / count(*),2) as Female
from Employees e;
```

Output:

Results | Messages

	Male	Female
1	50.670000000000	49.330000000000

16. Median salary calculation

```
select distinct
   PERCENTILE_CONT(0.5)
      within group (order by salary)
      over () as median_salary
from Salaries;
```

Output:

	median_salary
1	755000